



**ESCOLA DO
FUTURO**

CI/CD com GitHub Actions

Oficina 1 — Fundamentos e primeiros pipelines

Público: Iniciante / Intermediário · **Carga horária:** 4 horas (Oficina 1 de 2)

Robson Barbosa Souza

Escola do Futuro de Goiás - Paulo Renato de Souza

Agenda da Oficina 1

Bloco 1 — Fundamentos (2h)

1. O problema do software sem automação
2. Conceitos de CI e CD
3. Git e GitHub na prática
4. Visão geral do GitHub Actions

Bloco 2 — Mão na massa (2h)

5. Anatomia de um workflow
6. Primeiro pipeline real
7. Secrets, artefatos e cache
8. Boas práticas, erros comuns e exercício

Contexto: como o software chega ao usuário?

Antes da automação

- Código escrito localmente por cada dev
- Integração manual e esporádica
- Build e testes rodados "na máquina de alguém"

Depois da automação

- Pipeline reproduzível e padronizado
- Validação a cada commit
- Entrega previsível e rastreável

O problema: software sem automação

"Funciona na minha máquina."

- Integração tardia → conflitos e retrabalho
- Build e testes manuais → lentos e propensos a erro
- Deploy manual → risco e falta de rastreabilidade
- Ninguém sabe se o código está "verde" até o final

0 custo do problema

Sintoma	Consequência
Integração rara	Conflitos gigantes
Testes manuais	Bugs que escapam
Deploy manual	Erros de madrugada
Sem pipeline	Retrabalho constante

Efeito no time

↑ Tempo de entrega · ↑ Bugs em produção · ↓ Confiança

Bloco 1

Fundamentos de CI e CD

O que é Integração Contínua (CI)?

CI (Continuous Integration): integrar o código de todos os desenvolvedores em um repositório compartilhado, **várias vezes ao dia**, validando automaticamente cada integração.

- Build automático
- Testes automáticos (unitários, integração, lint)
- Feedback rápido: minutos, não dias
- O `main` deve estar sempre saudável

O que é Entrega Contínua (CD)?

CD (Continuous Delivery): garantir que o software possa ser **liberado para produção a qualquer momento**, de forma automatizada e confiável.

- Deploy automatizado em ambientes (staging, produção)
- Aprovação manual opcional antes de produção
- Rollback rápido e previsível

CI + CD = pipeline completo

Commit → Build → Teste → Deploy

CI + CD = pipeline completo

- CI cuida de: **integração, build e testes**
- CD cuida de: **empacotar, publicar e implantar**
- Juntos formam o **pipeline de entrega**

 Objetivo: entregar valor com segurança e frequência.

0 pipeline de entrega

Código



Build → Testes → Análise → Empacotar



Publicar artefato



Deploy (staging)



Deploy (produção)

Termos-chave

Termo	Significado
Build	Compilação/empacotamento do código
Teste	Validação automática de comportamento
Lint	Análise estática de qualidade
Artefato	Resultado pronto para deploy
Deploy	Implantação em um ambiente
Rollback	Voltar para uma versão anterior

Checkpoint – vamos refletir?

Qual é a diferença prática entre CI e CD?

Por que o `main` precisa estar sempre saudável?

O que acontece quando ninguém automatiza a integração?

Dica: responda com suas palavras antes de seguir.

Bloco 2

Git e GitHub na prática

Git: o essencial

Controle de versão distribuído

- `commit`: snapshot do código
- `branch`: linha independente de trabalho
- `merge` / `pull request`: integração revisada
- Histórico completo e rastreável

Comandos Git essenciais

```
git clone <url>           # copiar o repositório
git checkout -b feature   # criar branch
git add .                 # preparar alterações
git commit -m "mensagem" # registrar snapshot
git push origin feature   # enviar ao remoto
git pull                  # atualizar local
```



O GitHub Actions é **disparado** por esses eventos (push, PR).

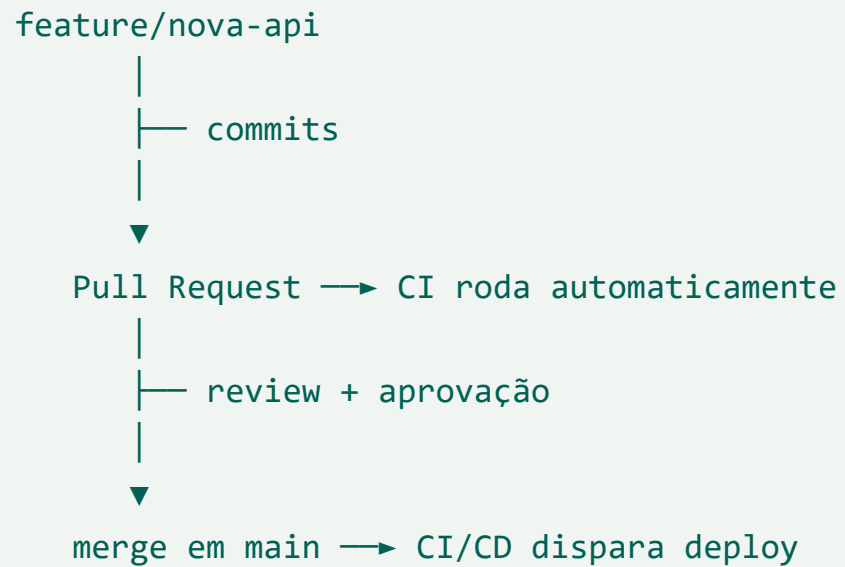
GitHub: plataforma de colaboração

- Repositórios remotos e versionados
- Pull Requests e code review
- Issues e projetos
- **GitHub Actions**: automação nativa
- Actions, Pages, Packages e muito mais



Na prática: o GitHub hospeda código **e** automação.

Fluxo com branches e PRs



Proteção de branches

Por que proteger o `main`?

- Impede merge direto sem revisão
- Exige CI verde antes do merge
- Garante rastreabilidade das mudanças

Configuração

Settings → Branches → Branch protection rules

 Sem proteção, qualquer push pode quebrar o `main`.

Pull Request na prática

1. Crie uma branch a partir do `main`
2. Faça commits com mensagens claras
3. Abra o PR descrevendo a mudança
4. CI roda automaticamente no PR
5. Revise, ajuste e aprove
6. Merge com CI verde

 O PR é o ponto de controle entre o código e o `main`.

Bloco 3

GitHub Actions: visão geral

O que é GitHub Actions?

Plataforma de **automação de fluxos de trabalho (workflows)** integrada ao GitHub, que executa tarefas em resposta a eventos do repositório.

- Gratuito para repositórios públicos
- Integração nativa com GitHub
- Ecossistema com milhares de ações prontas
- YAML declarativo, versionado junto com o código

Por que GitHub Actions?

Critério	GitHub Actions	CI tradicional
Configuração	YAML no repositório	Servidor dedicado
Custo inicial	Baixo	Alto
Integração	Nativa	Plugins
Manutenção	GitHub cuida	Sua equipe
Ecossistema	Marketplace	Variável

💡 Ideal para projetos que já vivem no GitHub.

Componentes do GitHub Actions

Componente	Papel
Workflow	Pipeline completo, definido em YAML
Event	Gatilho que dispara o workflow
Job	Conjunto de passos executados em um runner
Step	Comando ou ação individual
Runner	Máquina que executa os jobs
Action	Unidade reutilizável de código

 **Analogia:** o workflow é a receita; os steps são os passos; o runner é a cozinha.

Anatomia de um workflow

```
name: CI

on:
  push:
    branches: [ main ]

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "Hello, CI!"
```

Eventos (gatilhos) principais

Evento	Quando dispara
<code>push</code>	Commit enviado a um branch
<code>pull_request</code>	PR aberta/atualizada
<code>workflow_dispatch</code>	Manualmente (botão)
<code>schedule</code>	Em horário agendado (cron)
<code>release</code>	Publicação de release

Eventos na prática

```
on:  
  push:  
    branches: [ main ]  
  pull_request:  
    branches: [ main ]  
  workflow_dispatch:
```

Exemplo com schedule (cron)

```
on:  
  schedule:  
    - cron: "0 8 * * 1"    # toda segunda às 08:00
```

Jobs e Steps

Job

- Executa em um **runner** isolado
- Pode ter dependências (`needs`)
- Roda em paralelo por padrão

Step

- Comando (`run`) ou ação (`uses`)
- Executa em ordem sequencial
- Falha em um step → job falha

💡 Separe responsabilidades em jobs para ganhar paralelismo.

Runners: onde o código executa

GitHub-hosted

- Ubuntu, Windows, macOS
- Mantidos pelo GitHub
- `ubuntu-latest`, `windows-latest`, `macos-latest`

Self-hosted

- Sua própria infraestrutura
- Controle de hardware e rede
- Útil para requisitos específicos

⚠ Comece com runners hospedados. Self-hosted só quando houver necessidade real.

GitHub-hosted × Self-hosted

Aspecto	GitHub-hosted	Self-hosted
Manutenção	GitHub	Sua equipe
Custo	Minutos inclusos	Infra própria
Segurança	Ambiente isolado	Você controla
Personalização	Limitada	Total
Uso ideal	Começar	Requisitos especiais

Actions: unidades reutilizáveis

O que é uma Action?

Um bloco de código pronto para usar em steps.

Exemplos populares

- `actions/checkout` — baixa o repositório
- `actions/setup-node` — configura Node.js
- `actions/setup-python` — configura Python
- `actions/upload-artifact` — guarda arquivos
- `actions/cache` — acelera dependências

💡 Sempre use **versões fixas** (`@v4`), nunca `@main` em produção.

Bloco 4

Mão na massa: primeiros pipelines

Primeiro workflow na prática

Crie o arquivo `.github/workflows/ci.yml`:

```
name: CI

on: [push]

jobs:
  hello:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Olá, GitHub Actions!"
      - run: date
      - run: echo "Executado no runner ${ runner.os }"
```

Lendo a execução do workflow

Onde ver?

Actions (aba do repositório) → selecione o workflow

O que observar

- Status: sucesso  / falha 
- Logs de cada step
- Duração de cada job
- Artefatos gerados

 Cada step tem seu próprio log — use isso para debugar.

Exemplo real: build + testes (Node.js)

```
name: CI

on:
  push:
    branches: [ main ]
  pull_request:

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm
      - run: npm ci
      - run: npm run lint
      - run: npm test
```

Exemplo real: testes (Python)

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - run: pip install -r requirements.txt
      - run: pytest
```

Testando múltiplas versões (Matrix)

```
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        node-version: [18, 20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node-version }}
      - run: npm ci
      - run: npm test
```



O GitHub cria **um job por combinação** automaticamente.

Entendendo a Matrix

node-version	Job criado
18	test (18)
20	test (20)
22	test (22)

Quando usar?

- Suportar múltiplas versões de runtime
- Testar em SOs diferentes
- Validar compatibilidade de dependências

 Cuidado: cada combinação consome minutos de execução.

Dependências entre jobs (needs)

```
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Rodando testes"

  build:
    needs: test          # só roda se test passar
    runs-on: ubuntu-latest
    steps:
      - run: echo "Empacotando"
```



needs cria **ordem e dependência** entre jobs.

Secrets e variáveis

Secrets (Settings → Secrets and variables)

- Valores sensíveis: tokens, senhas, chaves
- Nunca aparecem em logs
- Acessados via `${{ secrets.NOME }}`

Variáveis de ambiente

- Configuração não sensível
- `env:` no workflow ou `${{ vars.NOME }}`

Secrets na prática

```
env:  
  NODE_ENV: production  
  
steps:  
  - run: deploy.sh  
    env:  
      API_TOKEN: ${ secrets.API_TOKEN }
```

Regras de ouro

- Nunca commitar secrets
- Use nomes claros e específicos
- Rotacione tokens periodicamente
- Limite o escopo de cada secret

Artefatos: compartilhando resultados

- Upload de arquivos gerados (builds, relatórios, logs)
- Download manual na página do workflow
- Compartilhamento entre jobs

```
- uses: actions/upload-artifact@v4
  with:
    name: build
    path: dist/
```

Artefatos: download entre jobs

```
jobs:
  build:
    steps:
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: build
          path: dist/

  deploy:
    needs: build
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: build
          path: dist/
      - run: ./deploy.sh
```

💡 Artefatos conectam o **build** ao **deploy**.

Cache: acelerando dependências

- Reutiliza dependências entre execuções
- `actions/cache` ou cache automático via `setup-*`

```
- uses: actions/cache@v4
  with:
    path: ~/.npm
    key: npm-${{ hashFiles('package-lock.json') }}
```

 Cache reduz tempo de CI e custo de minutos.

Ambientes e variáveis de ambiente

Environments (Settings → Environments)

- `development`, `staging`, `production`
- Regras de aprovação por ambiente
- Secrets específicos por ambiente

```
jobs:  
  deploy:  
    environment: production  
    runs-on: ubuntu-latest  
    steps:  
      - run: ./deploy.sh
```

 Ambientes organizam **onde** e **como** o deploy acontece.

Exercício prático

Desafio

Crie um workflow de CI para um projeto simples (Node ou Python):

1. Dispare em `push` e `pull_request`
2. Faça checkout do código
3. Configure a linguagem
4. Instale dependências
5. Rode lint e testes
6. Faça upload do resultado como artefato

Pergunta para reflexão

O que aconteceria se o teste falhar no pull request? E no main?

Bloco 5

Qualidade, boas práticas e erros comuns

Boas práticas (parte 1)

- Workflow versionado junto com o código
- Steps pequenos e com propósito único
- Use actions oficiais e versionadas (`@v4`)
- Proteja o `main` com branch protection + CI obrigatório
- Secrets nunca em código ou logs

Boas práticas (parte 2)

- Nomeie jobs e steps de forma clara
- Prefira `npm ci` a `npm install` (reprodutibilidade)
- Separe jobs para ganhar paralelismo
- Use `hashFiles` para chaves de cache precisas
- Documente o pipeline no README do repositório

Erros comuns

- ✗ Commit de secrets no repositório
- ✗ Testes que dependem da máquina local
- ✗ Workflow gigante e sem separação de jobs
- ✗ Ignorar falhas "intermitentes" do pipeline
- ✗ Rodar tudo em um único job sem paralelismo

💡 Um pipeline que falha de forma aleatória destrói a confiança do time.

Segurança no pipeline

- Secrets com escopo mínimo e rotação periódica
- Evite `pull_request_target` sem revisão cuidadosa
- Pinning de versões das actions
- Revisão de dependências (Dependabot)
- Permissões mínimas nos tokens (`permissions:`)

 O pipeline é parte da superfície de ataque do projeto.

Lendo logs e debugando falhas

Passos para debugar

1. Identifique o step que falhou (ícone )
2. Leia as últimas linhas do log
3. Reproduza o comando localmente
4. Corrija e faça push novamente

Dicas

- Use `echo` para inspecionar variáveis
- Ative debug com secrets `ACTIONS_STEP_DEBUG`
- Consulte a documentação oficial de cada action

Estratégias de teste no CI

Tipo	O que valida	Exemplo
Unitário	Funções isoladas	Jest, pytest
Integração	Módulos juntos	Supertest
Lint	Qualidade do código	ESLint, Ruff
Build	Compilação	tsc, webpack
E2E	Fluxo completo	Playwright

 Comece com unitário + lint; evolua conforme a maturidade.

O que o CI não resolve

CI automatiza, mas não substitui

- Revisão de código humana
- Arquitetura e design de qualidade
- Testes bem escritos
- Cultura de qualidade do time

⚠ Automação amplifica o que já existe: bons processos ficam ótimos; processos ruins ficam rapidamente ruins.

Checklist de um bom pipeline

- ☐ Roda em `push` e `pull_request`
- ☐ Checkout + setup da linguagem
- ☐ Instalação reprodutível de dependências
- ☐ Lint e testes automatizados
- ☐ Cache de dependências
- ☐ Artefatos para deploy
- ☐ Secrets protegidos
- ☐ Falhas visíveis e fáceis de debugar

Reflexão final

Como o GitHub Actions muda o dia a dia de um time?

Qual o primeiro workflow que você automatizaria no seu projeto?

O que diferencia um pipeline bom de um pipeline apenas "funcionando"?

Bloco 6

Síntese e próximos passos

O que aprendemos?

01

CI integra e valida o código continuamente

02

CD garante entrega automatizada e confiável

03

GitHub Actions = workflows YAML + eventos + runners

04

Secrets, artefatos e cache tornam o pipeline seguro e rápido

05

Boas práticas e proteção de branches sustentam a qualidade

Mapa mental do GitHub Actions

GitHub Actions

- ├ Eventos (push, PR, schedule...)
- ├ Workflow (YAML)
 - └ Jobs
 - └ Steps (run | uses)
- ├ Runners (ubuntu, windows, macos)
- ├ Actions (checkout, setup-*, cache...)
- ├ Secrets & variáveis
- └ Artefatos & cache

Próximos passos

Oficina 2 — Deploy contínuo e estratégias avançadas

- Deploy em ambientes (staging → produção)
- Environments e aprovações
- Estratégias de release e rollback
- Workflows reutilizáveis e composite actions
- Segurança avançada do pipeline



Pratique a Oficina 1 antes de avançar: pipeline é habilidade.

Referências

- GitHub Docs — GitHub Actions: <https://docs.github.com/actions>
- GitHub Docs — Workflow syntax:
<https://docs.github.com/actions/reference/workflow-syntax-for-github-actions>
- GitHub Actions Marketplace: <https://github.com/marketplace?type=actions>
- Jez Humble & David Farley — Continuous Delivery
- Gene Kim — The Phoenix Project